

Vector Rally - Relazione

Marta Musso

Luglio 2024

Contents

1	Introduzione	3
1.1	Regole del Gioco	3
2	Responsabilità Individuate e Assegnazioni	4
2.1	Model	4
2.1.1	Responsabilità	4
2.2	View	6
2.2.1	Responsabilità	6
2.3	Controller	7
2.3.1	Responsabilità	7
3	Principi SOLID e Pattern Utilizzati	8
3.1	Principi SOLID	8
3.1.1	Single Responsibility Principle (SRP)	8
3.1.2	Open/Closed Principle (OCP)	8
3.1.3	Liskov Substitution Principle (LSP)	9
3.1.4	Interface Segregation Principle (ISP)	9
3.1.5	Dependency Inversion Principle (DIP)	9
3.2	Pattern Utilizzati	9
3.2.1	Factory Pattern	9
3.2.2	Strategy Pattern	9
4	Strumenti Utilizzati	10
4.1	Gradle	10
4.2	Javadoc	11
4.3	JUnit Test	12
4.4	JavaFX	13
4.5	Scene Builder	13
4.6	GitHub	14
4.7	Costrutti Visti a Lezione	14

5	Istruzioni per Eseguire il Progetto e Link Utili	15
5.1	Run e Build	15
5.2	Link Utili	15
6	Implementazione della Specifica del Progetto	16
6.1	Simulazione da Terminale	16
6.2	Simulazione tramite Interfaccia Grafica	17
6.3	Istruzioni per le Simulazioni	17
7	Conclusione e sviluppi futuri	18
7.1	Sviluppi Futuri	18

Chapter 1

Introduzione

Vector Rally è un gioco di simulazione di corse automobilistiche a turni, progettato per essere giocato sia tramite interfaccia a riga di comando (CLI) che con una semplice interfaccia grafica (GUI) utilizzando JavaFX. Il gioco consente ai giocatori di controllare le loro auto su una pista, facendo uso di algoritmi per determinare le mosse valide e gestendo le interazioni tra i giocatori e la pista.

Il gioco si svolge su tracciati rappresentati da file di testo (`.txt`), in cui ogni simbolo corrisponde a una specifica componente della pista. Ad esempio, i simboli possono rappresentare la linea di partenza e quella di arrivo.

1.1 Regole del Gioco

Il gioco segue le seguenti regole fondamentali:

- **Obiettivo:** Vince il primo giocatore che raggiunge la linea di arrivo.
- **Eliminazione:** Un giocatore senza mosse valide viene eliminato dalla partita.
- **Mosse:** Ad ogni turno, i giocatori possono muovere le loro auto in base alle mosse valide determinate dall'algoritmo. Le mosse valide sono determinate dalla posizione attuale dell'auto e dalla configurazione del tracciato.

Chapter 2

Responsabilità Individuate e Assegnazioni

Il progetto Vector Rally è stato sviluppato seguendo il pattern Model-View-Controller (MVC). Questo approccio facilita la separazione delle responsabilità, migliorando la manutenibilità e l'estensibilità del codice. Di seguito vengono dettagliate le responsabilità principali e le rispettive classi e interfacce che le implementano.

2.1 Model

Il modello contiene la logica del gioco e rappresenta i dati con cui l'applicazione lavora. Comprende le classi che definiscono i giocatori, le auto, le mosse, le posizioni, i componenti della pista e le regole del gioco.

2.1.1 Responsabilità

- **Definire i Giocatori:** Gestire le informazioni e le azioni dei giocatori, inclusi i loro attributi e comportamenti.
 - `Player` - Interfaccia per i giocatori.
 - `HumanPlayer` - Rappresenta un giocatore umano.
 - `BotPlayer` - Rappresenta un giocatore bot.
 - `DefaultPlayer` - Classe astratta per un giocatore di default.
- **Gestire le Auto:** Rappresentare le auto dei giocatori, inclusi colori, stati e movimenti.
 - `Car` - Interfaccia per le auto.

- `RaceCar` - Implementa un'auto da corsa.
 - `CarColour` - Enumera i colori delle auto.
- **Gestire i Movimenti:** Definire e validare i movimenti delle auto e le loro posizioni sulla pista.
 - `Move` - Record che rappresenta un movimento.
 - `Position` - Rappresenta una posizione sulla pista.
 - `Acceleration` - Rappresenta l'accelerazione di un'auto.
 - `Coordinates` - Interfaccia per le coordinate di una posizione.
 - `Direction` - Enumera le direzioni di un movimento.
 - `Shift` - Interfaccia per uno spostamento.
 - `Vector` - Interfaccia per un vettore di movimento.
- **Definire i Componenti del Tracciato:** Rappresentare i vari componenti della pista, come la linea di partenza, la linea di arrivo, le strade e gli ostacoli.
 - `TrackComponent` - Enumera i componenti del tracciato.
 - `RaceTrack` - Rappresenta la pista da corsa.
 - `Track` - Interfaccia per il tracciato.
 - `Component` - Rappresenta un componente generico del tracciato.
- **Implementare le Regole del Gioco:** Stabilire le regole per il gioco, compresa la validazione delle mosse e l'interazione con i componenti del tracciato.
 - `BasicMoveValidator` - Valida i movimenti base.
 - `BasicMovesGenerator` - Genera i movimenti base.
 - `MoveValidator` - Interfaccia per la validazione delle mosse.
- **Implementare Algoritmi di Movimento:** Definire e implementare algoritmi per generare mosse basate sulle posizioni e i tracciati.
 - `AStar` - Implementa l'algoritmo A* per la ricerca del percorso.
 - `NeighborsGenerator` - Interfaccia per generare i vicini di una posizione.
 - `EightNeighborsGenerator` - Implementa un generatore di vicini su otto direzioni.

- `FourNeighborsGenerator` - Implementa un generatore di vicini su quattro direzioni.
- `Node` - Rappresenta un nodo nell'algoritmo A*.
- **Definire Strategie per i Bot:** Stabilire le strategie decisionali per i bot.
 - `BotStrategy` - Enumera le strategie dei bot.
 - `DecisionStrategy` - Interfaccia per le strategie decisionali dei bot.
 - `EasyBotStrategy` - Implementa una strategia facile per i bot.
 - `MediumBotStrategy` - Implementa una strategia media per i bot.
 - `HardBotStrategy` - Implementa una strategia difficile per i bot.

2.2 View

La vista è responsabile della presentazione dei dati all'utente. Include le classi che gestiscono l'input e l'output per le interfacce CLI e GUI.

2.2.1 Responsabilità

- **Gestione dell'Input/Output:** Raccogliere input dall'utente e visualizzare messaggi e dati di gioco.
 - `IOController` - Interfaccia per definire le principali azioni di I/O.
 - `CLIIIOController` - Interfaccia per l'I/O della CLI.
 - `GraphicalIOController` - Implementazione dell'I/O per la GUI.
 - `TerminalIOController` - Implementazione dell'I/O per la CLI.
- **Presentare il Tracciato e le Informazioni di Gioco:** Visualizzare la pista, le posizioni delle auto e le informazioni relative al gioco, come i turni e le vittorie.
 - `TerminalUtils` - Implementazione delle utilità per la CLI.
- **Fornire Messaggi di Gioco:** Generare e visualizzare messaggi contestuali per informare i giocatori sulle azioni di gioco e gli stati.
 - `MessageProvider` - Fornisce i messaggi di gioco.
 - `GameMessageProvider` - Implementazione dei messaggi di gioco.
 - `Output` - Interfaccia per l'output dei messaggi.

2.3 Controller

Il controller gestisce il flusso del gioco e le interazioni dell'utente con il sistema. Include le classi che orchestrano le operazioni di setup e gestione del match.

2.3.1 Responsabilità

- **Gestione e Arbitraggio del Match:** Controllare e gestire le condizioni e le conseguenze per le eliminazioni, le vittorie e la gestione dei turni.
 - `MatchController` - Interfaccia per il controllo del match.
 - `CLIMatchController` - Implementazione del controllo del match per la CLI.
 - `GUIMatchController` - Implementazione del controllo del match per la GUI.
- **Setup della Partita:** Gestire la creazione del setup della partita, inclusa la lista dei giocatori e il tracciato creato.
 - `GameSetup` - Interfaccia per il setup del gioco.
 - `GUIGameSetup` - Implementazione del setup del gioco per la GUI.
 - `CLIGameSetup` - Implementazione del setup del gioco per la CLI.
 - `SetupResult` - Record risultante dal setup del gioco.
- **Costruzione del Tracciato:** Costruire un tracciato, matrice di componenti, a partire da un file `.txt`.
 - `RaceTrackBuilder` - Costruisce il tracciato da un file.
- **Gestione delle Strategie dei Bot:** Definire e assegnare le strategie ai bot.
 - `BotStrategyFactory` - Factory per le strategie dei bot.

Chapter 3

Principi SOLID e Pattern Utilizzati

In questo capitolo vengono analizzati i principi SOLID seguiti durante lo sviluppo del progetto Vector Rally, insieme ai pattern di progettazione utilizzati.

3.1 Principi SOLID

I principi SOLID sono linee guida che aiutano a mantenere il codice flessibile e manutenibile. Di seguito sono descritti i principi SOLID con esempi tratti dal progetto Vector Rally.

3.1.1 Single Responsibility Principle (SRP)

Ogni classe dovrebbe avere una sola responsabilità, ovvero una sola ragione per cambiare.

- `RaceCar` è responsabile solo della rappresentazione di un'auto da corsa.
- `BasicMoveValidator` ha la sola responsabilità di validare i movimenti, assicurandosi che siano conformi alle regole del gioco.

3.1.2 Open/Closed Principle (OCP)

Le classi dovrebbero essere aperte per l'estensione ma chiuse per la modifica.

- `BotStrategyFactory` può essere estesa per aggiungere nuove strategie senza modificare il codice esistente.

- `RaceTrack` può essere estesa per aggiungere nuovi tipi di componenti del tracciato senza alterare la struttura base.

3.1.3 Liskov Substitution Principle (LSP)

Le classi derivate devono poter essere sostituite alle classi base senza alterare la correttezza del programma.

- `BotPlayer` e `HumanPlayer` estendono entrambe `Player` e possono essere utilizzate in qualsiasi contesto dove sia prevista un'istanza di `Player`.

3.1.4 Interface Segregation Principle (ISP)

Le interfacce dovrebbero essere specifiche per il client, evitando di creare interfacce grandi e generiche.

- `Vector`, `Coordinates` e `Shift` sono interfacce separate per gestire lo spostamento da una posizione a un'altra.

3.1.5 Dependency Inversion Principle (DIP)

Le dipendenze dovrebbero essere invertite rispetto alle astrazioni.

- `CLIGameEngine` dipende da `GameSetup` per la configurazione del gioco, consentendo la sostituzione delle implementazioni senza modificare il motore del gioco.

3.2 Pattern Utilizzati

Nel progetto Vector Rally sono stati utilizzati alcuni pattern di progettazione per migliorare la struttura del codice, rendendolo più flessibile ed estensibile.

3.2.1 Factory Pattern

Il Factory Pattern è utilizzato per creare oggetti senza esporre la logica di creazione al client. Questo pattern è impiegato nella classe `BotStrategyFactory` per creare le strategie dei bot.

3.2.2 Strategy Pattern

Il Strategy Pattern è utilizzato per definire una famiglia di algoritmi, incapsularli e renderli intercambiabili. Questo pattern è utilizzato per determinare le strategie decisionali dei bot.

Chapter 4

Strumenti Utilizzati

In questo capitolo vengono descritti gli strumenti utilizzati durante lo sviluppo del progetto Vector Rally. Ogni sezione fornisce una panoramica dello strumento e degli esempi di codice per illustrarne l'uso.

4.1 Gradle

Gradle è stato utilizzato per gestire le dipendenze e automatizzare il processo di build del progetto. Grazie a Gradle, è stato possibile configurare facilmente le librerie necessarie e gestire le task di compilazione, esecuzione e test.

Esempio di file `build.gradle`:

```
plugins {  
    id 'application'  
    id 'org.openjfx.javafxplugin' version '0.1.0'  
    id("buildlogic.java-application-conventions")  
}  
  
repositories {  
    mavenCentral()  
}  
  
run {  
    standardInput = System.in  
}  
  
dependencies {  
    implementation 'org.apache.commons:commons-text:1.10.0'}
```

```

        implementation 'org.openjfx:javafx-controls:18.0.1'
        implementation 'org.openjfx:javafx-fxml:18.0.1'
        implementation project(':api')
    }

    java {
        toolchain {
            languageVersion = JavaLanguageVersion.of(21)
        }
    }

    javafx {
        version = "22.0.1"
        modules = [ 'javafx.controls', 'javafx.fxml' ]
    }

    application {
        mainClass = 'it.unicam.cs.mpmgc.vetorally.app.GraphicalApp'
    }

```

4.2 Javadoc

Javadoc è stato utilizzato per documentare il codice. Generando la documentazione HTML direttamente dal codice sorgente, è possibile mantenere la documentazione sempre aggiornata con l'evoluzione del codice.

Esempio di commento Javadoc:

```

/**
 * Retrieves the component at the specified position on the track.
 *
 * @param x the x coordinate.
 * @param y the y coordinate.
 * @return the track component at the specified position.
 */
TrackComponent getComponentAt(int x, int y);

```

4.3 JUnit Test

JUnit è stato utilizzato per testare il codice. Grazie a JUnit, è possibile scrivere test automatizzati per verificare la correttezza delle funzionalità implementate e garantire che le modifiche al codice non introducano bug.

Esempio di test JUnit:

```
public class BotPlayerTest {

    private BotPlayer botPlayer;
    private Car car;
    private BotStrategy strategy;

    @BeforeEach
    void setUp() {
        car = new RaceCar(CarColour.BLUE);
        strategy = BotStrategy.EASY;
        botPlayer = new BotPlayer(car, strategy);
    }

    @Test
    void constructorShouldThrowExceptionWhenCarIsNull() {
        assertThrows(NullPointerException.class,
            () -> new BotPlayer(null, strategy));
    }

    @Test
    void constructorShouldThrowExceptionWhenStrategyIsNull() {
        assertThrows(NullPointerException.class,
            () -> new BotPlayer(car, null));
    }

    @Test
    void getStrategyShouldReturnStrategy() {
        assertEquals(strategy, botPlayer.getStrategy());
    }
}
```

4.4 JavaFX

JavaFX è stato utilizzato per creare l'interfaccia grafica del gioco. Fornisce una ricca libreria di componenti UI e strumenti per costruire interfacce utente moderne e interattive.

Esempio di utilizzo di JavaFX:

@FXML

```
public void initialize() throws Exception {
    setupButton.setDisable(true);
    loadTrackNames();
    setupDifficultyMenu();
    setupShiftRuleMenu();
    trackMenuButton.setOnAction(event -> checkReadyStatus());
    difficultyMenuButton.setOnAction(event -> checkReadyStatus());
    shiftRuleMenuButton.setOnAction(event -> checkReadyStatus());
    setupButton.setOnAction(event -> {
        try {
            handleSetupButtonAction(event);
        } catch (IOException e) {
            e.printStackTrace();
        }
    });
}
```

4.5 Scene Builder

Scene Builder è stato utilizzato per progettare le interfacce utente in JavaFX. Permette di costruire le scene in modo visuale e di generare il file FXML corrispondente, facilitando la separazione tra logica e presentazione.

Esempio di codice FXML generato con Scene Builder:

```
<rowConstraints>
    <RowConstraints minHeight="10.0" prefHeight="30.0" />
    <RowConstraints minHeight="10.0" prefHeight="30.0" />
    <RowConstraints minHeight="10.0" prefHeight="30.0" />
</rowConstraints>
```

4.6 GitHub

GitHub è stato utilizzato per il controllo della versione e dello sviluppo del progetto nei suoi vari livelli di implementazione. Grazie a GitHub, è possibile tenere traccia delle modifiche e gestire i rami di sviluppo.

In particolare, sono stati utilizzati 3 rami differenti per gestire lo sviluppo del progetto nelle sue fasi differenti:

- `main`
- `development_base`
- `development_GUI`

4.7 Costrutti Visti a Lezione

Durante lo sviluppo del progetto, sono stati utilizzati diversi costrutti e concetti visti a lezione, come l'uso delle interfacce e dei record.

Esempio di record:

```
public record SetupResult(  
    NeighborsGenerator generator ,  
    RaceTrack raceTrack ,  
    List<Player> players) {  
}
```

Esempio di Generics:

```
public class BasicMovesGenerator<T extends NeighborsGenerator> {  
    private final T neighborsGenerator ;  
    private final MoveValidator moveValidator ;  
}
```

Chapter 5

Istruzioni per Eseguire il Progetto e Link Utili

5.1 Run e Build

Per costruire ed eseguire il progetto, utilizzare i seguenti comandi:

```
gradle build
gradle run
```

Verrà mostrata l'esecuzione dell'interfaccia grafica che, come specificato nel capitolo 6, non comprende tutte le funzionalità e le finezze incluse nella simulazione giocabile da linea di comando. Per eseguire la simulazione da terminale, eseguire `TerminalApp` del modulo `app` o modificare il file `build.gradle` del modulo `app` come segue:

```
application {
    mainClass = 'it.unicam.cs.mpmgc.vectorrally.app.GraphicalApp'
}

application {
    mainClass = 'it.unicam.cs.mpmgc.vectorrally.app.TerminalApp'
}
```

5.2 Link Utili

Viene reso disponibile il riferimento al [Github Repository](#) contenente il codice del progetto Vector Rally.

Chapter 6

Implementazione della Specifica del Progetto

Il progetto Vector Rally consente di eseguire la simulazione di gioco sia da terminale sia tramite interfaccia grafica, come descritto nel Capitolo 5.

6.1 Simulazione da Terminale

La simulazione da terminale è stata curata in molti aspetti, prevedendo anche l'interazione di un utente umano. Questa versione offre molte funzionalità aggiuntive, come la possibilità di selezionare le difficoltà differenti per ogni singolo bot oppure uniche per tutti. La simulazione da terminale è stata realizzata per essere completa e sufficientemente dettagliata, occupando la maggior parte del tempo di realizzazione del progetto.



Figure 6.1: Sconfitta durante un match sul terminale

6.2 Simulazione tramite Interfaccia Grafica

Al contrario, l'interfaccia grafica non è altrettanto soddisfacente. Per ragioni di capacità, conoscenze, tempo e priorità, la versione GUI fornisce solo una simulazione di gioco molto modesta. Questa parte del progetto rappresenta più che altro il tentativo di iniziare a imparare ad utilizzare strumenti come JavaFX e comprenderne i principali aspetti.

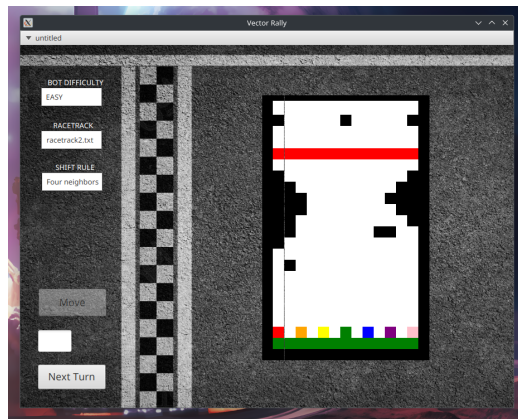


Figure 6.2: Setup iniziale del match su interfaccia grafica

6.3 Istruzioni per le Simulazioni

Per entrambe le simulazioni, è sufficiente seguire le istruzioni che vengono mostrate man mano. La simulazione da terminale fornirà i prompt necessari per interagire con il gioco, mentre l'interfaccia grafica presenterà i controlli visivi necessari per eseguire la simulazione.

In sintesi, la versione terminale di Vector Rally rappresenta l'implementazione principale e più completa del progetto, mentre la versione grafica è un primo passo verso l'apprendimento e l'utilizzo di JavaFX e della programmazione ad eventi.

Chapter 7

Conclusione e sviluppi futuri

Il codice è stato progettato per essere estensibile e modulare, facilitando l'aggiunta di nuove funzionalità e miglioramenti futuri.

7.1 Sviluppi Futuri

Il progetto Vector Rally è stato sviluppato con l'obiettivo di creare un codice modulare e aperto alle estensioni. Questo approccio permette di aggiungere nuove funzionalità senza dover modificare significativamente il codice esistente, garantendo una maggiore flessibilità e manutenibilità. Tra gli sviluppi futuri previsti ci sono:

- **Estensione dei Controller:** Migliorare e ampliare le funzionalità dei controller per la GUI, per gestire in modo più efficace le varie interazioni di gioco.
- **Interfaccia Grafica Avanzata:** Rendere l'interfaccia grafica più avanzata e gradevole, includendo tutte le funzionalità attualmente gestite dalla versione terminale. Questo include la gestione delle difficoltà selezionabili per i bot, la visualizzazione dettagliata della pista e delle auto, e l'interazione utente più intuitiva.
- **Miglioramenti nell'Interazione Utente:** Implementare nuove funzionalità che migliorino l'esperienza utente, come animazioni fluide, feedback visivi per le azioni degli utenti e una navigazione più semplice.